

BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 2 Roll no: 13 PRN No: 12410783

Subject Name : Deep Learning

Name of Student: Sarthak Anil Patkar

Problem Statement - Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

1. Algorithm: Tomato Leaf Disease Classification Using CNN

Objective

To classify tomato leaf images into their respective disease categories using a CNN trained on the Tomato Leaf Disease dataset.

Algorithm

Input: A tomato leaf image

Output: Predicted disease class and prediction confidence

1. Download the Dataset

- Download the Tomato Leaf Disease dataset using KaggleHub.
- Locate the training and validation directories.

2. Load and Preprocess Images

- Read images from their respective class folders.
- Resize each image to **128 × 128 pixels**.
- Divide the dataset into training and validation sets.
- Assign numerical labels to each disease class.

3. Apply Data Augmentation

- Randomly flip images horizontally.
- Apply small random rotations.
- Apply random zooming.
- This helps the model generalize to variations in leaf images.

4. Normalize Pixel Values

- Convert pixel values from the range **0–255 to 0–1**.

5. Construct the CNN

- Apply four convolutional layers with filters:
 - 32 filters
 - 64 filters
 - 128 filters
 - 256 filters
- Apply ReLU activation after each convolution.
- Apply max pooling after every convolutional block.
- Flatten the extracted features.
- Use a dense layer with 128 neurons.
- Apply dropout of 0.5 to reduce overfitting.
- Use a softmax output layer to predict the disease class.

6. Compile the Model

- Optimizer: Adam
- Learning rate: 0.001
- Loss function: Sparse Categorical Crossentropy
- Evaluation metric: Accuracy

7. Train the Model

- Train using the training dataset.
- Validate using the validation dataset.
- Train for up to 12 epochs.
- Reduce the learning rate when validation loss stops improving.

8. Evaluate the Model

- Calculate validation loss and accuracy.
- Generate a classification report.
- Generate a confusion matrix.
- Compare actual and predicted classes.

9. Save the Model

- Save the trained model as `tomato_disease_cnn.keras`.

10. Predict a New Image

- Resize and preprocess the input image.
- Pass it through the CNN.
- Select the class with the highest softmax probability.
- Display the predicted disease and confidence.

Code:

```
# Install KaggleHub if required
```

```
!pip install -q kagglehub
```

```
import os
```

```
import random
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
import kagglehub
```

```
import seaborn as sns
```

```
from tensorflow.keras import layers, models
```

```
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
from sklearn.metrics import (
```

```
    classification_report,
```

```
    confusion_matrix
```

```
)
```

```
# Reproducibility
```

```
SEED = 42
```

```
random.seed(SEED)
```

```
np.random.seed(SEED)
```

```
tf.random.set_seed(SEED)
```

```
# Download dataset
```

```
path = kagglehub.dataset_download(
    "kaustubhb999/tomatoleaf"
)

print("Dataset downloaded at:", path)

# Dataset directories
dataset_root = os.path.join(path, "tomato")

train_dir = os.path.join(dataset_root, "train")
val_dir = os.path.join(dataset_root, "val")

print("Training directory:", train_dir)
print("Validation directory:", val_dir)

if not os.path.exists(train_dir):
    raise FileNotFoundError("Training directory not found")

if not os.path.exists(val_dir):
    raise FileNotFoundError("Validation directory not found")

IMG_SIZE = 128
BATCH_SIZE = 32
EPOCHS = 12
LEARNING_RATE = 0.001

# Load training dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
```

```
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=SEED
)

# Load validation dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=False
)

# Class names
class_names = train_ds.class_names
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)

IMG_SIZE = 128
BATCH_SIZE = 32
EPOCHS = 12
LEARNING_RATE = 0.001

# Load training dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir,
```

```

    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=True,
    seed=SEED
)

# Load validation dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=False
)

# Class names
class_names = train_ds.class_names
num_classes = len(class_names)

print("Classes:", class_names)
print("Number of classes:", num_classes)

# Data augmentation
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])

# Optimize input pipeline

```

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
train_ds = train_ds.cache().shuffle(1000).prefetch(  
    buffer_size=AUTOTUNE  
)
```

```
val_ds = val_ds.cache().prefetch(  
    buffer_size=AUTOTUNE  
)
```

```
model = models.Sequential([
```

```
    # Input layer
```

```
    layers.Input(  
        shape=(IMG_SIZE, IMG_SIZE, 3)  
    ),
```

```
    # Data augmentation
```

```
    data_augmentation,
```

```
    # Normalize pixels from 0-255 to 0-1
```

```
    layers.Rescaling(1.0 / 255),
```

```
    # Convolutional Block 1
```

```
    layers.Conv2D(  
        32,  
        (3, 3),  
        activation="relu",  
        padding="same"
```

```
),
```

```
layers.MaxPooling2D((2, 2)),
```

```
# Convolutional Block 2
```

```
layers.Conv2D(
```

```
    64,
```

```
    (3, 3),
```

```
    activation="relu",
```

```
    padding="same"
```

```
),
```

```
layers.MaxPooling2D((2, 2)),
```

```
# Convolutional Block 3
```

```
layers.Conv2D(
```

```
    128,
```

```
    (3, 3),
```

```
    activation="relu",
```

```
    padding="same"
```

```
),
```

```
layers.MaxPooling2D((2, 2)),
```

```
# Convolutional Block 4
```

```
layers.Conv2D(
```

```
    256,
```

```
    (3, 3),
```

```
        activation="relu",
        padding="same"
    ),

    layers.MaxPooling2D((2, 2)),

    # Classification layers
    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    # Reduce overfitting
    layers.Dropout(0.5),

    # Output layer
    layers.Dense(
        num_classes,
        activation="softmax"
    )
])

model.summary()

optimizer = tf.keras.optimizers.Adam(
    learning_rate=LEARNING_RATE
)
```

```
model.compile(
    optimizer=optimizer,
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Reduce learning rate when validation loss stops improving
reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=2,
    min_lr=1e-6,
    verbose=1
)

# Train model
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=[reduce_lr]
)

# Training statistics
actual_epochs = len(history.history["loss"])

best_epoch = (
    np.argmax(history.history["val_accuracy"]) + 1
```

```
)
```

```
best_val_accuracy = max(  
    history.history["val_accuracy"]  
)
```

```
best_val_loss = min(  
    history.history["val_loss"]  
)
```

```
print("Actual epochs:", actual_epochs)
```

```
print("Best epoch:", best_epoch)
```

```
print(  
    f"Best validation accuracy: "  
    f"{best_val_accuracy * 100:.2f}%"  
)
```

```
print(  
    f"Best validation loss: "  
    f"{best_val_loss:.4f}"  
)
```

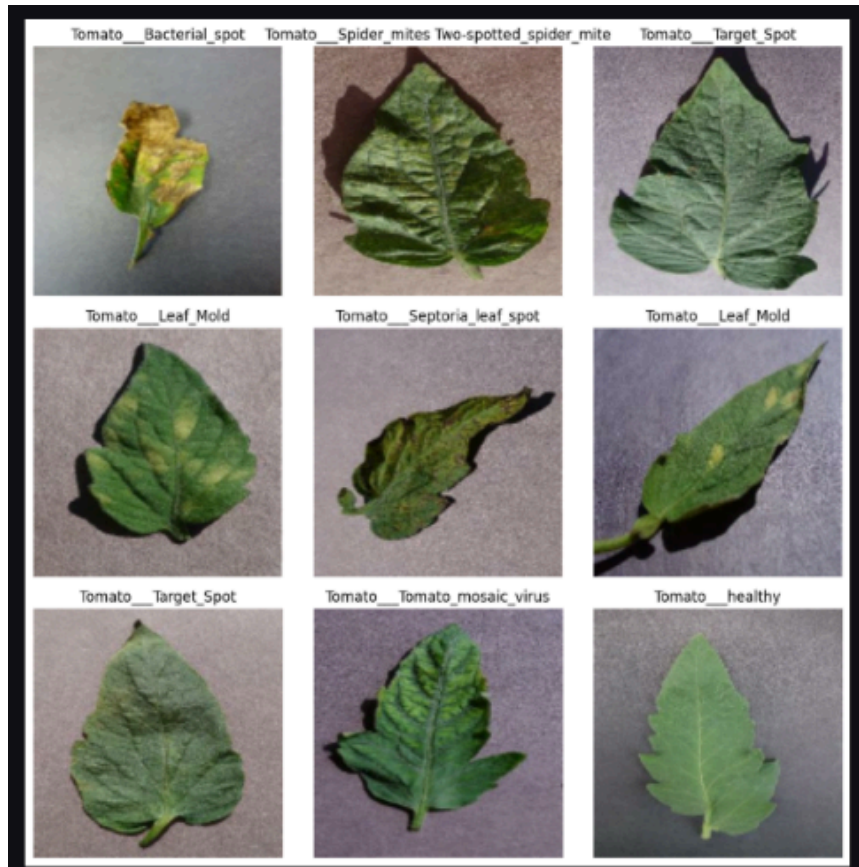
```
plt.figure(figsize=(8, 5))
```

```
plt.plot(  
    history.history["accuracy"],  
    marker="o",  
    label="Training Accuracy"  
)
```

```
plt.plot(
    history.history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.grid()
plt.show()
```

Output:



```
=====
CNN MODEL ARCHITECTURE
=====
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
sequential_2 (Sequential)	(None, 128, 128, 3)	0
rescaling_2 (Rescaling)	(None, 128, 128, 3)	0
conv2d_8 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_8 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_9 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_9 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_10 (Conv2D)	(None, 32, 32, 128)	73,856
max_pooling2d_10 (MaxPooling2D)	(None, 16, 16, 128)	0
conv2d_11 (Conv2D)	(None, 16, 16, 256)	295,168
max_pooling2d_11 (MaxPooling2D)	(None, 8, 8, 256)	0
flatten_2 (Flatten)	(None, 16384)	0
dense_4 (Dense)	(None, 128)	2,097,280
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 10)	1,290

Total params: 2,486,986 (9.49 MB)

Trainable params: 2,486,986 (9.49 MB)

Non-trainable params: 0 (0.00 B)

Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 94.5%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 80.9%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 100.0%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 100.0%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 92.7%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 99.9%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Target_Spot
Confidence: 62.0%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 99.9%



Actual: Tomato__Bacterial_spot
Predicted: Tomato__Bacterial_spot
Confidence: 100.0%

